+Тестовое задание нужно выложить на GitHub

Написать сервис, который по расписанию забирает среднюю цену торговой пары (пару можете выбрать сами или взять по умолчанию BNBUSDT) у биржи Binance, сохраняет её в БД и отдаёт через REST.

Ссылка на описание API (данные можно брать по запросу в https://api.binance.com/api/v3/avgPrice). Внимательно ознакомьтесь с возможными ошибками и с блоком Rate limits.

Как это работает

Приложение получает информацию из https://api.binance.com/api/v3/avgPrice по расписанию и сохраняет ее в БД.

1) Первый endpoint должен в качестве ответа отдавать информацию о последней сохраненной записи (наиболее актуальная информация). В ответе должна содержаться следующая информация:

- symbol — символ торговой ПАРЫ, по умолчанию можно запрашивать BNBUSDT
- price — цена торговой пары
- fetchedAt — время получения записи от стороннего API в формате ISO-8601 в UTC (YYYY-MM-DDTHH:mm:ssZ)

2) Второй endpoint должен выдавать рассчитанную на основании имеющихся в шлю сервисе данных информацию о средней за цене торговой пары за определенный промежуток времени [from, to - формат дат на ваше усмотрение]. Пользователь должен будет иметь возможность получить информацию за указанный период.

```
Request
GET /api/v1/prices/average?from=...&to=...

Response
{
“averagePrice”: 10.2
}
```

3) Третий endpoint должен выдавать записи, сохраненные в интервале [from, to - формат дат на ваше усмотрение] с пагинацией, к примеру:

```
GET /api/v1/prices?from=...&to=...&page=0&size=20
Response:
{
    "items": [
        {
            "symbol": "BNBUSDT",
```

```
            "price": "...",
            "fetchedAt": "..."
        }
    ],
    "page": 0,
    "size": 20,
    "totalElements": 135,
    "hasNext": true
}
```

4) Четвертый endpoint должен произвести вычисление тренда по значениям в таблице в интервале [from, to - формат дат на ваше усмотрение], который приходит в запросе:

```
GET /api/v1/prices/trend?from=...&to=...
```

И вернуть ответ:

```
Response
{
  "symbol": "BNBUSDT",
  "start_date": "2026-09-01",
  "end_date": "2026-09-30",
  "trend": "UPWARD"
}
```

Возможные значения параметра "trend" на ваше усмотрение (как пример можно использовать UPWARD, DOWNWARD, FLAT).

Взаимодействие пользователя с приложением происходит через REST API.

Требования

- Java 17+, Spring Boot 3.x, PostgreSQL, Liquibase или Flyway для схемы БД.
- Интервал опроса для шедуллера настраивается через переменную окружения BINANCE_POLL_INTERVAL (через ISO-8601 - PT30S).
- Желательно обернуть это все в докер compose с возможностью быстрого поднятия и запуска вашего сервиса.
- Обработка ошибок и логирование: сбой обращения к Binance не должен останавливать расписание и не должен приводить к дублированию или потере данных в БД.
- Работа с деньгами без double (BigDecimal или на ваше усмотрение).
- Тесты: unit-тесты бизнес-логики и минимум один интеграционный тест (Testcontainers и/или WireMock на ваше усмотрение).
- README: как запустить, как вызвать эндпоинты, какие переменные окружения поддерживаются.
- Чистый код: понятные имена, адекватное разделение слоёв, без на ходу выдуманных значений.

- При использовании LLM важно, чтобы вы могли объяснить каждую строчку своего проекта (как, для чего и почему то или иное решение в вашем коде лучше в конкретной ситуации).